BỘ BÀI TẬP THỰC HÀNH

Structured Output và thuật toán

4 tình huống gần với công việc doanh nghiệp: chăm sóc khách hàng, vận hành kho, đối soát chi phí và quản lý dự án.

01	Ưu tiên yêu cầu khách hàng
02	Tìm giờ nhận xe tại kho
03	Rà soát hóa đơn gửi trùng
04	Xếp việc theo thứ tự trước - sau

GenAI đọc và điền dữ liệu. Code kiểm tra và tính kết quả.

Nội dung

Bốn bài tập có thể làm riêng hoặc ghép thành một ứng dụng. Mỗi bài đi từ dữ liệu đầu vào đến kết quả có thể kiểm tra được.

Bài 01 - Khách hàng nào cần được hỗ trợ trước? 3

Bài 02 - Xe giao hàng nên vào cửa kho lúc mấy giờ? 5

Bài 03 - Hóa đơn này đã được gửi trước đó chưa? 6

Bài 04 - Việc nào phải làm trước? 7

Giới thiệu và cách làm

Mục tiêu

Hãy tưởng tượng bạn nhận một email khách hàng, một yêu cầu giao hàng hoặc một biên bản họp. GenAI đọc nội dung đó và điền vào một mẫu JSON có các ô rõ ràng. Sau đó, code của bạn kiểm tra dữ liệu và tính kết quả: việc nào làm trước, xe nào được vào kho lúc nào, chứng từ nào cần xem lại.

Structured Output là phần GenAI điền đúng các ô trong mẫu. **Thuật toán** là phần code dùng dữ liệu đã kiểm tra để đưa ra kết quả. GenAI không tự quyết định thứ tự xử lý hay tự kết luận một chứng từ là gian lận.

Các tình huống dưới đây mô phỏng việc doanh nghiệp thường gặp. Dữ liệu và quy tắc trong bài là dữ liệu học tập, không thuộc một doanh nghiệp cụ thể.

Bài	Bạn sẽ làm gì?	Cách xử lý chính	Độ khó gợi ý
01	Chọn yêu cầu khách hàng cần xử lý trước	Hàng đợi ưu tiên	Vừa
02	Tìm giờ nhận xe còn trống	Gộp khoảng thời gian	Vừa
03	Tìm chứng từ có thể đã được gửi trước đó	Tra cứu bằng khóa	Dễ-vừa
04	Xếp việc theo thứ tự trước-sau	Đồ thị phụ thuộc	Khó

Yêu cầu chung cho mỗi bài

1. Trang web cho người dùng nhập nội dung, xem GenAI đã đọc ra những gì, sửa thông tin sai, rồi bấm xác nhận để xem kết quả và lý do.
2. Backend yêu cầu GenAI trả đúng mẫu dữ liệu (schema). Trường chưa biết để **null**; các lựa chọn như mức độ khẩn cấp chỉ được lấy trong danh sách cho phép. Bạn có thể dùng Pydantic và **with_structured_output** như đã học.

3. Code **kiểm tra lại** kết quả GenAI: ngày giờ có hợp lệ không, ID có tồn tại không, số tiền có âm không, dữ liệu có bị trùng không. JSON đúng khuôn vẫn có thể chứa thông tin sai.
4. Lưu nội dung gốc, JSON GenAI trả về, bản người dùng sửa, kết quả cuối và thời gian xử lý để có thể xem lại. Giữ API key ở backend.
5. Có dữ liệu mẫu để chạy thử mà không tốn tiền gọi model. Test phần thuật toán riêng, không phụ thuộc GenAI.
6. Chỉ dùng dữ liệu giả trong bài học; không gửi chứng từ hoặc thông tin cá nhân thật lên dịch vụ AI khi chưa được phép.

Nộp bài: code trang web và backend, **README** hướng dẫn chạy, mẫu dữ liệu GenAI phải trả, ít nhất 3 test cho thuật toán (ca thường, ca sát giới hạn, dữ liệu sai), ảnh hoặc video ngắn cho luồng chính. Viết thêm vài câu giải thích GenAI làm phần nào và code thuật toán làm phần nào.

Chấm điểm (100): GenAI trả đúng mẫu và dữ liệu được kiểm tra 25; thuật toán đúng và đủ nhanh 30; trang web và API 20; test 15; cách giải thích và trải nghiệm sử dụng 10. Có thể làm từng bài độc lập; không cần ghép cả bốn vào một app.

Đáp án mẫu: code và hướng dẫn chạy.

Những việc tương tự đang có ở doanh nghiệp

- Bộ phận chăm sóc khách hàng cần đưa yêu cầu tới đúng người và xử lý kịp hạn: tài liệu Zendesk.
- Kho hàng cần hẹn giờ xe đến và sắp cửa nhận hàng: tài liệu SAP.
- Bộ phận tài chính cần nhận ra chứng từ được gửi nhiều lần để tránh hoàn tiền hai lần: tài liệu Oracle.
- Nhóm dự án cần biết công việc nào chặn công việc khác: tài liệu Jira.

Bài 01 - Khách hàng nào cần được hỗ trợ trước?

Một buổi sáng, nhóm chăm sóc khách hàng nhận hàng chục tin nhắn. Có người hỏi cách đổi mật khẩu, có người báo bị tính tiền hai lần, có người không thanh toán được. Nếu chỉ xử lý theo thứ tự tin nhắn đến, một vấn đề rất gấp có thể bị bỏ lại phía sau.

Việc bạn cần làm: xây một trang web nhận tin nhắn, đọc ra nội dung chính, rồi xếp các yêu cầu thành hàng đợi để nhân viên biết nên xử lý cái nào trước. Mỗi yêu cầu gọi là một **ticket**. Hệ thống phải giải thích được vì sao ticket đó đứng đầu.

Bước 1 - GenAI đọc tin nhắn

Hệ thống đã biết mã ticket (**ticket_id**) và thời điểm nhận (**received_at**). GenAI đọc lời khách viết và điền các thông tin còn lại vào mẫu JSON này:

```
{
  "ticket_id": "T01",
  "intent": "billing",
  "severity": "high",
  "product": "checkout",
  "summary": "Khách bị tính tiền hai lần",
  "needs_human_review": false
}
```

intent cho biết khách hỏi về thanh toán (**billing**), lỗi kỹ thuật (**technical**), tài khoản (**account**) hay việc khác (**other**). **severity** là mức độ khẩn cấp: **low**, **medium**, **high**, **critical**. **product** có thể là **null** nếu khách không nói đến sản phẩm nào. **summary** dài tối đa 200 ký tự. Nếu tin nhắn mơ hồ, dùng **intent=other** và **needs_human_review=true**. **ticket_id** phải đúng mã hệ thống đã cấp.

Mỗi mức khẩn cấp có một **hạn xử lý** (SLA). Code tự tính hạn từ lúc nhận: **critical** trong 30 phút, **high** trong 2 giờ, **medium** trong 8 giờ, **low** trong 24 giờ. GenAI không tự đoán hạn này.

Bước 2 - Code xếp hàng đợi

Hãy nghĩ đến quầy lấy số thứ tự, nhưng ai sắp trễ hạn sẽ được gọi trước. Dùng **min-heap** để giữ ticket có hạn gần nhất ở đầu. Nếu hai ticket cùng hạn, ưu tiên mức khẩn cấp cao hơn; nếu vẫn bằng nhau, ưu tiên tin nhắn đến trước; nếu vẫn bằng nhau, so mã ticket theo alphabet. Khóa so sánh là **(deadline_at, -severity_rank, received_at, ticket_id)** với **critical=4**, **high=3**, **medium=2**, **low=1**.

Người dùng phải thêm được ticket, lấy ticket kế tiếp và xem **top K** ticket đầu hàng đợi. Việc xem không được xóa ticket. Nếu mã ticket bị trùng, hãy từ chối hoặc nêu rõ quy tắc cập nhật.

Ví dụ: coi GenAI đã đọc đúng và nhân viên đã xác nhận ba ticket sau.

ID	Nhận lúc	Severity	Deadline tính ra
T01	2026-10-21 09:00 +07:00	high	11:00
T02	2026-10-21 09:20 +07:00	critical	09:50
T03	2026-10-21 08:30 +07:00	medium	16:30

Kết quả cần ra: **T02 → T01 → T03**, vì T02 hết hạn lúc 09:50. Top 2 là **T02, T01**. Nếu thêm **T04** mức **high** nhận lúc 07:00, hạn của nó là 09:00 nên T04 lên đầu. Khi đã có **n** ticket trong heap, thêm hoặc lấy một ticket cần khoảng **O(log n)** bước.

Trang web cần có

- Màn hình nhập tin nhắn và hàng đợi có cột hạn xử lý, mức khẩn cấp, tóm tắt, lý do xếp hạng.
- API tối thiểu: **POST /tickets/extract**, **POST /tickets/confirm**, **GET /tickets/queue?limit=...**, **POST /tickets/next**.
- Nếu GenAI trả mức khẩn cấp ngoài danh sách, sai ID hoặc không tính được hạn xử lý, báo lỗi để người dùng kiểm tra; chưa đưa ticket đó vào hàng đợi.

Tự kiểm tra

1. Thứ tự và top K đúng với dữ liệu mẫu; **top K** không làm thay đổi lần lấy ticket kế tiếp.
2. Hai ticket cùng hạn và mức khẩn cấp vẫn có thứ tự rõ ràng nhờ thời điểm nhận, rồi mã ticket.
3. Thử mã trùng, thời gian không có múi giờ và mức khẩn cấp sai. App phải báo rõ vấn đề.

Nếu muốn làm thêm: mỗi nhân viên có kỹ năng và số việc tối đa khác nhau. Hãy chỉ ra ticket nào chưa có nhân viên phù hợp để nhận.

Bài 02 - Xe giao hàng nên vào cửa kho lúc mấy giờ?

Kho có nhiều cửa để xe vào giao hàng, nhưng mỗi cửa chỉ phục vụ được một xe tại một thời điểm. Nhà vận chuyển gửi email: “Xe tôi đến từ 9 giờ, cần một tiếng để dỡ hàng.” Nhân viên kho muốn thấy ngay cửa nào còn trống sớm nhất, thay vì dò từng dòng trên lịch.

Việc bạn cần làm: xây trang web đọc email đặt lịch, tìm một khoảng giờ phù hợp và tránh để hai xe cùng được đặt vào một cửa.

Bước 1 - GenAI đọc email

GenAI điền nội dung email vào mẫu sau:

```
{
  "request_id": "R01",
  "carrier": "ACME Logistics",
  "earliest_start": "2026-10-21T09:00:00+07:00",
  "duration_minutes": 60,
  "dock_type": "standard",
  "missing_fields": []
}
```

earliest_start là giờ sớm nhất xe có thể đến, phải có múi giờ. **duration_minutes** là số phút cần dùng cửa kho, phải lớn hơn 0. **dock_type** là cửa thường (**standard**) hoặc cửa lạnh (**refrigerated**). Nếu email không nói cần bao lâu, để **duration_minutes=null**, ghi **missing_fields=["duration_minutes"]** và hỏi lại người dùng. Đừng tự đoán thời lượng từ loại hàng. Backend kiểm tra mã yêu cầu và giờ làm việc.

Bước 2 - Code tìm giờ trống

Với mỗi cửa đúng loại, hãy xếp các lịch đã có theo giờ bắt đầu. Những lịch giao nhau hoặc nối sát nhau thì gộp thành một khoảng bận. Chẳng hạn 09:00-10:00 và 10:00-11:00 nghĩa là cửa bận liên tục từ 09:00 đến 11:00.

Từ giờ xe có thể đến, tìm khoảng trống đầu tiên đủ dài trong giờ mở cửa 08:00-18:00. Chọn giờ bắt đầu sớm nhất; nếu hai cửa cùng giờ, chọn mã cửa đứng trước theo alphabet. Dùng khoảng **[start, end)**, nên một xe rời cửa lúc 10:00 và xe sau vào lúc 10:00 là hợp lệ. Khi người dùng bấm xác nhận, kiểm tra lại lịch mới nhất rồi mới lưu, vì có thể người khác vừa đặt cùng giờ. Với **n** lịch đã có, mục tiêu là **O(n log n)** để sắp xếp và gộp.

Ví dụ: ngày 21/10/2026 có hai cửa thường **A**, **B**.

Cửa	Booking đã có
A	09:00-10:00; 10:00-11:00
B	09:30-10:15

Yêu cầu **R01** muốn đến từ 09:00 và cần 60 phút. Cửa A bận đến 11:00; cửa B trống từ 10:15. **Kết quả cần ra: B, 10:15–11:15**. Nếu một xe khác hỏi lại cùng giờ sau khi R01 đã đặt, kết quả phải đổi. Nếu không còn chỗ trong ngày, trả **no_slot**; không tự chuyển sang hôm sau.

Trang web cần có

- Cho nhập email, xem/sửa thông tin GenAI đọc ra và xem lịch từng cửa.
- API tối thiểu: **POST /requests/extract**, **POST /slots/suggest**, **POST /bookings/confirm**.
- Khi xác nhận, kiểm tra lại xung đột trên dữ liệu mới nhất để tránh hai người cùng đặt một giờ.

Tự kiểm tra

1. Ví dụ trên trả về cửa B lúc 10:15; hai lịch nối sát nhau được xem như một khoảng bận.
2. Lịch cũ kết thúc đúng lúc lịch mới bắt đầu không bị báo trùng.
3. Thử thiếu thời lượng, sai múi giờ, giờ kết thúc trước giờ bắt đầu, hết chỗ và hai người cùng đặt một giờ. App phải xử lý rõ từng trường hợp.

Nếu muốn làm thêm: thêm cửa lạnh hoặc 15 phút dọn cửa giữa hai xe. Khi đó khoảng bận phải tính cả thời gian dọn cửa.

Bài 03 - Hóa đơn này đã được gửi trước đó chưa?

Nhân viên đi công tác và gửi hóa đơn để công ty hoàn tiền. Vài ngày sau, cùng hóa đơn có thể được gửi thêm lần nữa qua email hoặc bản chụp khác. Người duyệt có nhiều chứng từ nên khó nhớ đã thấy hóa đơn nào.

Việc bạn cần làm: xây trang web đọc thông tin trên chứng từ và đánh dấu những bản **có thể** đã được gửi trước đó. Hệ thống chỉ giúp người duyệt kiểm tra; không tự kết luận nhân viên gian lận hay tự từ chối hoàn tiền.

Bước 1 - GenAI đọc chứng từ

Nếu đầu vào là ảnh hoặc PDF, hãy lấy văn bản bằng OCR trước. GenAI đọc văn bản đó và điền vào mẫu:

```
{
  "submission_id": "E01",
  "vendor": "ABC Travel",
  "invoice_number": "INV-0042",
  "transaction_date": "2026-10-19",
  "amount_minor": 1250000,
  "currency": "VND",
  "receipt_id": "R-42",
  "missing_fields": []
}
```

vendor là nơi bán, **invoice_number** là số hóa đơn, **receipt_id** là mã biên nhận. **amount_minor** là số tiền ở đơn vị nhỏ nhất: với VND là đồng, với USD là cent. Lưu bằng số nguyên không âm để tránh sai số khi so sánh. Trường không đọc được để **null** và ghi tên vào **missing_fields**. Backend kiểm tra mã, ngày, tiền và loại tiền.

Bước 2 - Code tìm bản có thể trùng

Hãy tạo một "nhãn tra cứu" cho mỗi chứng từ. Nếu có mã biên nhận, nhãn chính là loại tiền + mã biên nhận. Nếu có tên nơi bán và số hóa đơn, tạo thêm nhãn từ loại tiền + tên nơi bán + số hóa đơn; nhãn này giúp tìm được bản gửi sau bị thiếu mã biên nhận. Dùng **hash map** để tìm nhanh chứng từ cũ theo các nhãn đó.

Trước khi so sánh, bỏ dấu cách thừa, chuẩn hóa Unicode và không phân biệt chữ hoa/thường ở tên nơi bán, số hóa đơn. **Không tự xóa dấu gạch nối:** **INV-0042** và **INV0042** có thể là hai mã khác nhau.

- Tìm thấy nhãn cũ: **duplicate_candidate**, cho biết chứng từ cũ là bản nào và các trường khác nhau ở đâu.
- Không đủ thông tin để tạo nhãn: **needs_review**. Chỉ cùng nơi bán, số tiền và ngày **chưa đủ** để gọi là trùng.
- Chưa thấy nhãn này: **unique_under_current_rules**. Nghĩa là "chưa trùng theo quy tắc hiện tại", không phải "chắc chắn hợp lệ".

Ví dụ:

ID	Vendor	Invoice	Receipt	Số tiền VND
E01	**ABC Travel**	**INV-0042**	**R-42**	1.250.000
E02	**abc travel**	**inv-0042**	**R-42**	1.250.000
E03	**ABC Travel**	null	null	1.250.000

Kết quả cần ra: E01 chưa trùng theo quy tắc; E02 là ứng viên trùng E01 dù chữ hoa và dấu cách khác nhau; E03 thiếu mã nên cần người xem lại. Nếu E02 có số tiền khác, vẫn đánh dấu vì mã biên nhận trùng và hiển thị chênh lệch. Với **n** chứng từ, cách tra cứu này cần khoảng **O(n)** thời gian trung bình.

Trang web cần có

- Cho tải chứng từ mẫu hoặc nhập văn bản đã OCR, xem/sửa thông tin GenAI đọc ra, rồi xem cặp chứng từ nghi trùng. Người duyệt có nút xác nhận hoặc bỏ nghi vấn.
- API tối thiểu: **POST /expenses/extract**, **POST /expenses/check**, **POST /expenses/review**.
- Lưu thông tin trước và sau khi người dùng sửa. Nếu sửa mã biên nhận, phải kiểm tra trùng lại.

Tự kiểm tra

1. Ba chứng từ mẫu cho ra ba trạng thái như trên.
2. Chữ hoa và dấu cách thừa không làm mất trùng; **INV-0042** và **INV0042** không bị coi là một mã nếu không có mã biên nhận trùng.
3. Thử thiếu mã, tiền âm và loại tiền không hỗ trợ. App không được trả "chưa trùng" khi dữ liệu chưa đủ tin cậy.

Nếu muốn làm thêm: tìm cả mã hóa đơn gần giống do OCR đọc sai. Chỉ đưa kết quả vào danh sách cần xem lại, vì hai hóa đơn khác nhau có thể trông rất giống nhau.

Bài 04 - Việc nào phải làm trước?

Sau một buổi họp, nhóm có nhiều việc phải làm: chốt API, duyệt giao diện, tích hợp frontend, kiểm thử. Biên bản thường viết thành đoạn văn, nên người mới vào nhóm khó biết việc nào có thể bắt đầu ngay và việc nào phải chờ.

Việc bạn cần làm: xây trang web đọc biên bản, tạo danh sách việc và chỉ ra thứ tự làm hợp lý. Nếu kế hoạch nói "A chờ B" nhưng đồng thời "B chờ A", app phải báo vòng phụ thuộc để nhóm sửa.

Bước 1 - GenAI đọc biên bản

GenAI tìm hai thứ: **task** (công việc) và **dependency** (việc nào phải xong trước việc nào). Mỗi task có mã ID riêng; không dùng tên việc làm mã vì tên có thể bị sửa hoặc trùng.

```
{
  "tasks": [
    {"id": "A", "title": "Chốt yêu cầu API", "owner": "Product"},
    {"id": "B", "title": "Duyệt thiết kế UI", "owner": "Design"},
    {"id": "C", "title": "Tích hợp frontend", "owner": "Engineering"},
    {"id": "D", "title": "Kiểm thử end-to-end", "owner": "QA"}
  ],
  "dependencies": [
    {"before": "A", "after": "C", "evidence": "Sau khi chốt API mới tích hợp frontend"},
    {"before": "B", "after": "C", "evidence": "Duyệt UI trước khi tích hợp"},
    {"before": "C", "after": "D", "evidence": "Kiểm thử sau khi tích hợp"}
  ]
}
```

owner là người hoặc nhóm phụ trách, có thể để **null** nếu biên bản không nói. **evidence** là một câu ngắn trong biên bản cho thấy vì sao có quan hệ trước-sau. GenAI không được tự thêm quan hệ khi không có bằng chứng. Trước khi tính thứ tự, người dùng phải được xem và sửa danh sách. Backend từ chối mã ID lạ, ID trùng và việc tự phụ thuộc chính nó; quan hệ bị ghi lặp thì chỉ tính một lần.

Bước 2 - Code xếp thứ tự công việc

Hãy hình dung mỗi công việc là một ô, mũi tên **A → B** nghĩa là phải xong A rồi mới làm B. Dùng **topological sort** (Kahn hoặc DFS) để xếp các ô sao cho mọi mũi tên đều đi từ việc trước sang việc sau. Nếu có nhiều việc cùng có thể bắt đầu, chọn mã ID theo alphabet để mỗi lần chạy cho cùng một kết quả.

Nếu không thể xếp hết, nghĩa là có vòng chờ nhau. Trả **cycle_detected** và chỉ ra ít nhất một vòng hoặc các quan hệ liên quan để người dùng sửa. Đừng hiển thị nửa danh sách như thể đó là kế hoạch hoàn chỉnh. Với **V** công việc và **E** quan hệ, mục tiêu là **O(V + E)**, hoặc **O((V + E) log V)** nếu dùng heap để chọn ID theo alphabet.

Kết quả cần ra với ví dụ trên: A → B → C → D. A và B đều có thể làm đầu tiên; ta chọn A trước vì ID đứng trước B. Nếu thêm quan hệ **D → A**, sẽ có vòng **A → C → D → A**, nên app phải báo **cycle_detected**.

Trang web cần có

- Cho nhập biên bản, xem/sửa danh sách việc và quan hệ trước-sau, rồi hiển thị thứ tự hoặc vòng cần sửa.
- API tối thiểu: **POST /plans/extract**, **POST /plans/validate**, **POST /plans/order**.
- Chỉ dùng quan hệ đã được người dùng xác nhận; hiển thị câu làm bằng chứng bên cạnh mỗi quan hệ.

Tự kiểm tra

1. Ví dụ trên cho cùng kết quả dù đổi thứ tự các task trong JSON.
2. Thêm **D → A** thì báo vòng; xóa quan hệ đó thì lập được thứ tự.
3. Thử mã lạ, task tự chờ chính nó, ID trùng, quan hệ trùng và kế hoạch không có quan hệ nào. App phải xử lý rõ ràng.

Nếu muốn làm thêm: nhóm những việc có thể làm cùng lúc. Chưa có thời lượng và số người thì đừng tự biến thứ tự thành lịch ngày/giờ.